Today
Welcome to your next lesson: Challenge: queues.

I'm preparing your personalized lesson based on what you've learned so far, what's next for you, and the best way for you to practice.

Let’s dive in — you’re making real progress.

Ron, let's turn this into a quick queue game 🎮

We’ll do 6 short rounds about queues (FIFO: first in, first out). I’ll give one small task at a time, you answer, and we move on. No long typing needed.

Let’s start super simple, with real life, not code:

Imagine a line at a food truck:

Alice arrives
Then Bob
Then Carol

The truck serves from the front of the line.

Who gets served first?

You can just answer with the name.

Alice

☑ Correct — Alice is first in, because she entered the line first. That’s FIFO in real life. 🚚

Now let’s move to a tiny bit of Python with a queue using a list.

We start with an empty queue:

Python

```
queue = []
queue.append("A")   # step 1
queue.append("B")   # step 2
queue.append("C")   # step 3
removed = queue.pop(0)  # step 4
```

In step 4 we remove from the front with pop(0).

What is the value of removed after this code runs?

Just answer with a single letter.

"A"

Nice, yes — removing from the front gives "A" 🎯

Round 3 now, still with lists.

We start with:

Python

```
from collections import deque

q = deque()
q.append("X")    # 1
q.append("Y")    # 2
q.append("Z")    # 3
left = q.popleft()  # 4
```

popleft() removes from the front of the queue.

What is the value of left after this runs?

Answer with a single letter.

"x"

Perfect — left is "X" (case doesn't matter here) ☑

Round 4: list-based queue again 💪

Python

```
queue = []
queue.append(1)      # step 1
queue.append(2)      # step 2
queue.append(3)      # step 3
first = queue.pop(0) # step 4
queue.append(4)      # step 5
second = queue.pop(0)  # step 6
```

What is the value of second after all this?

Just answer with a single number.

2

Nice work, Ron — you nailed these queue challenges 🔥

Quick recap:

You correctly used FIFO thinking in real-life (lines) and in Python.
You tracked both list-based queues (append, pop(0)) and deque-based queues (append, popleft) without getting mixed up.
You followed how the front of the queue moves as we enqueue and dequeue.
You've got the core queue behavior down. 🎯

If you're still feeling fresh, you can jump to the next lesson in Learn → Data structures & algorithms → Stacks and queues.
If you want, tell me how queues feel so far:

👍 Clear
😐 Kind of ok but fuzzy
😵 Still confusing
I can stick around and reinforce things with more custom examples either way.

How would you rate this lesson?

totally clear loong forward to sortig with them later

Love hearing that, Ron 👀 — that's exactly where I want you: queues feel natural, and you're already thinking ahead to sorting and more advanced uses.

You've officially completed the queues challenge for this lesson 🎓

Very short summary of what you own now:

You can spot FIFO behavior in real life (lines, tasks, jobs).
You know how to use:
List queues: append + pop(0)
deque queues: append + popleft
You can mentally trace what's at the front and what's at the back after several operations.
Next strong move from here 💪:

In the Learn tab, continue to the next lesson in Data structures & algorithms → Stacks and queues.
When you start seeing sorting or more complex problems with queues, I can help you:
walk through them step by step,
or turn them into visual diagrams so you can see what's going on.

Before you jump:

Do you want a tiny warm-up on "how queues might interact with sorting later" (just a 1-paragraph intuition), or are you heading straight to the next lesson?

sure can do an intro is ben 30 years since learning them in col,

Love that you're coming back to this after 30 years — that takes real grit 💪

Here's a tiny, no-math intro to how queues can connect to sorting later:

Think of sorting as "put these items in order,"

and a queue as "handle these items one by one, in arrival order."

In some algorithms, you:

Put items into a queue as they become "ready" to be processed.
Repeatedly dequeue (front item), look at it, maybe place it into the correct spot in a growing sorted structure, and then move on to the next.

So the queue's job is often:

not to sort itself,
but to control the order in which work is done while some other logic decides the sorted order.

You'll see this pattern a lot later in algorithms:

queue = flow of work,

sorting = how results are arranged.

If you'd like, next time we can take a small real-world example (like sorting customers by priority while they still arrive FIFO) and sketch it visually so you see both ideas at once.

For now, you’re all set to continue in the Learn tab to your next lesson.

yes

You’re all set on this queues challenge, Ron 🎓🔥

Quick closing snapshot:

You understand FIFO deeply (both real-world and in Python).
You’re comfortable with:
List queues: append + pop(0)
deque queues: append + popleft
You can trace what happens to the front and back as items move through.

You also have a good instinct for how queues can support bigger algorithms (like sorting) by controlling the flow of work.

Next step:

In the Learn tab, continue to the next lesson in Data structures & algorithms → Stacks and queues.
Any time something feels fuzzy — even if it’s “I learned this 30 years ago but forgot” — just tell me, and I’ll walk it with you slowly, step by step, with visuals. 🍸

When you start the next topic, you can ping me with:

“Maestro, walk me through this slowly.”

and I’ll sync right up with where you are.